

Universidad Autónoma de Baja California

Facultad de Contaduría y Administración



Carrera: Licenciatura de Inteligencia en negocios

Materia: Patrones de Comportamiento de Datos.

3.3.4 Analytical Module: Dynamic Evasion in Adversarial Systems

Elaborado por:

[Contreras Nevarez Carlos Manuel] – [01281541] – [carlos.contreras7@uabc.edu.mx]

Docente: Adrian Rodriguez Aguiñaga.

07 de Octubre del 2026

Tijuana, B.C., México

Reporte de evasión dinámica en sistemas adversariales

Esta práctica analiza la navegación de un agente en un laberinto con enemigos móviles. El sistema recalcula mediante búsqueda en anchura (BFS) una ruta hacia la meta y excluye las celdas ocupadas por enemigos y sus zonas de peligro. Las evidencias aportadas muestran una ejecución local que alcanzó la meta en el paso 62 con semilla 42. El análisis explica el costo de recalcular, los bloqueos por desconexión y una propuesta probabilística de intersección de trayectorias.

Objetivo y procedimiento

El objetivo fue integrar la evasión de enemigos en la búsqueda de rutas y visualizar el comportamiento del agente. Se trabajó en Windows con Python 3.11, un entorno virtual `bfs_env` y Jupyter Notebook. La preparación empleó NumPy, Matplotlib e IPython para representar la matriz y mostrar la animación; `collections.deque` y `random` corresponden a la biblioteca estándar de Python.

El escenario utiliza una matriz de 31×31 , paredes perimetrales, obstáculos interiores y cinco enemigos. El agente parte de (1, 1) y busca llegar a (29, 29), usando cuatro movimientos cardinales. En cada paso, los enemigos realizan movimientos aleatorios válidos y el agente consulta una ruta considerando sus nuevas posiciones. Si existe un siguiente nodo, avanza una celda; cuando no existe ruta, permanece en su posición.

La zona de peligro de cada enemigo comprende su celda y las cuatro celdas vecinas cardinales. La subrutina base permite entrar a la meta aunque esté dentro de una zona de peligro y parte de la posición actual del agente incluso si esta queda amenazada. Estas excepciones deben considerarse al interpretar la seguridad del sistema: la evasión espacial no garantiza anticipar todos los movimientos futuros.

Ejecución local

La evidencia de Jupyter identifica Python 3.11.9 y el ejecutable `D:\practica_bfs\bfs_env\Scripts\python.exe`. La captura de terminal muestra la creación del entorno con `py -3.11 -m venv bfs_env`. También incluye un intento fallido de consultar la versión por un error en la opción escrita; esto no impidió crear el entorno. Las capturas de preparación se conservan en el anexo.

Resultados y evidencia de la simulación

La figura 1 muestra el estado final de la ejecución. El título registra “Evasión dinámica con BFS — Paso 62” y “Meta alcanzada | Semilla 42”. Se distinguen los muros, los enemigos, sus zonas de peligro y la meta. Esta captura respalda el resultado final de esa corrida, pero por sí sola no demuestra cuántas veces cambió la ruta ni cuánto tiempo consumió cada búsqueda.

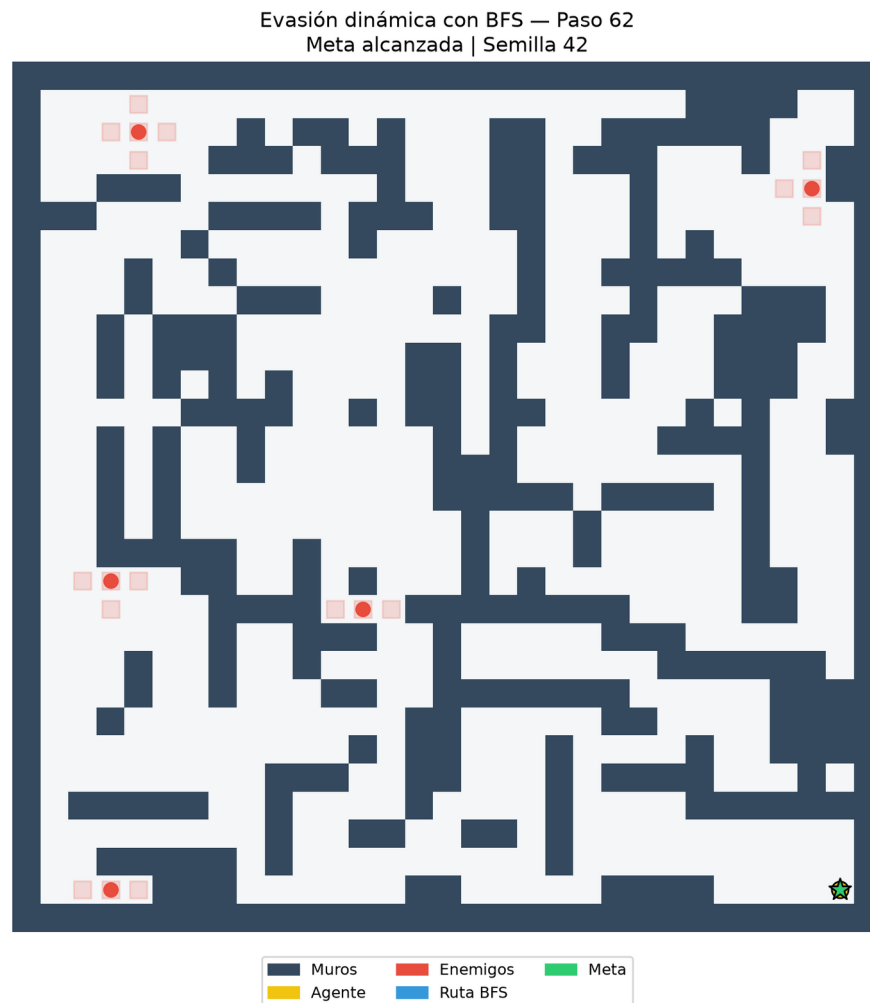


Figura 1. Estado final aportado por el alumno. El agente alcanza la meta en el paso 62 con semilla 42. Fuente: ejecución local del alumno.

El video de demostración es el medio solicitado para observar la evolución temporal y el desvío de la trayectoria ante los enemigos. El notebook conserva la implementación y permite revisar la integración de la subrutina. No se incluyen mediciones de tiempo ni pruebas sobre una cuadrícula de 1000×1000 ; su viabilidad se discute de forma teórica.

Análisis de la complejidad de BFS

En BFS estándar, cada vértice se marca al descubrirse y se incorpora a la cola como máximo una vez. Al extraerlo se examina su lista de vecinos; cada arista se revisa un número constante de veces. Con una cola eficiente y un registro de predecesores, el costo de una búsqueda está acotado por una constante multiplicada por el número de vértices y aristas (Cormen et al., 2009).

$$T_{\text{BFS}} \leq C \cdot (|V| + |E|)$$

Si se ejecuta una búsqueda en cada uno de k fotogramas, se suman los costos de las k búsquedas. Tomando $|V|$ y $|E|$ como tamaños máximos durante la simulación, y una constante C independiente del fotograma, se obtiene:

$$T_{\text{total}} \leq \sum_{t=1}^k C \cdot (|V| + |E|)$$

$$T_{\text{total}} \leq C \cdot k \cdot (|V| + |E|)$$

$$T_{\text{total}} = O(k \cdot (|V| + |E|))$$

En una cuadrícula $N \times N$ con cuatro vecinos por celda, hay como máximo N^2 vértices y $2N(N - 1)$ aristas no dirigidas antes de quitar obstáculos. Por tanto, $|E| = O(|V|)$ y la cota se simplifica a $O(k \cdot N^2)$. En una cuadrícula de 1000×1000 existen hasta 1 000 000 de vértices y 1 998 000 aristas. Reexplorar una región de ese tamaño en cada fotograma puede consumir una parte considerable del presupuesto de ejecución y memoria en Python. La posibilidad de operar en tiempo real depende de la implementación, el equipo y la frecuencia requerida; debe comprobarse mediante mediciones.

D* Lite y LPA* reutilizan información de búsquedas anteriores y reparan los efectos de los cambios en el grafo. Pueden reducir el trabajo cuando las modificaciones son locales, como ocurre al desplazarse un enemigo y cambiar unas pocas celdas bloqueadas. D* Lite está orientado a la replanificación durante el desplazamiento del agente. No garantizan que toda actualización sea pequeña: cambios que afectan corredores esenciales pueden propagar reparaciones a una región amplia (Koenig & Likhachev, 2002).

La cota anterior requiere distinguir el algoritmo estándar de la implementación base: path + [neighbor] copia la ruta completa para cada nodo descubierto. Si L es la longitud máxima de una ruta copiada, una cota conservadora es $O(|E| + |V| \cdot L)$ por búsqueda. Guardar únicamente predecesores y reconstruir la ruta al finalizar evita este costo adicional. Si se realizan dos búsquedas por paso para comparar rutas, el factor constante se duplica, sin cambiar la cota asintótica del BFS estándar.

Encierro topológico y congelamiento

Los cuellos de botella son pasos estrechos que conectan distintas regiones del laberinto. Cuando los enemigos y sus buffers ocupan todos los pasos necesarios entre la posición del agente y la meta, forman un corte de vértices. Al excluir esas celdas, el grafo de navegación queda desconectado: la meta ya no pertenece a la componente alcanzable desde el agente, aunque exista una conexión en el laberinto sin enemigos.

$$G_{\text{seguro}}(t) = G[V - H_t]$$

H representa las celdas temporalmente prohibidas. Esta expresión describe la exclusión general; la subrutina base mantiene las excepciones de la posición inicial y la meta. El bloqueo debe evaluarse sobre el grafo que realmente utiliza el programa.

BFS explora la componente accesible y termina sin encontrar la meta. La función devuelve [start] y la condición $\text{len}(\text{path}) > 1$ impide mover al agente. El congelamiento es consecuencia de la desconexión y de esa regla de control. En el siguiente paso se vuelve a buscar; si los enemigos liberan el corredor, el movimiento puede reanudarse.

BFS minimiza la cantidad de pasos hacia una meta definida. No incorpora un objetivo de maximizar la separación respecto de los enemigos. Por eso, cuando falla la búsqueda, no elige automáticamente una posición alternativa de refugio. Una estrategia de respaldo podría escoger una celda vecina permitida que aumente la distancia al enemigo más cercano, considerando además la conectividad y evitando callejones sin salida. Esta mejora es una propuesta y no se presenta como una función verificada en las evidencias.

Reflexión sobre el comportamiento

Alcanzar la meta en una corrida muestra que el sistema logró resolver ese escenario, pero no garantiza éxito para todas las semillas. Los enemigos pueden bloquear corredores, aproximarse a la posición actual del agente o modificar la accesibilidad de la meta. Para evaluar robustez sería necesario repetir la simulación y registrar éxito, colisiones, pasos inmóviles y duración de las búsquedas. No se asignan valores a estas métricas porque las capturas aportadas no los contienen.

Modelo de Markov en tiempo continuo

Se propone un modelo aproximado con tres estados: S indica separación segura; R, proximidad de riesgo; y C, intersección o colisión. C es absorbente porque registra que el encuentro ya ocurrió. Su matriz generadora Q contiene tasas por unidad de tiempo, no probabilidades (Whitt, 2006).

Q	S	R	C
S	$-\alpha$	α	0
R	β	$-(\beta + \gamma)$	γ
C	0	0	0

α es la tasa de pasar de separación segura a riesgo; β es la tasa de regresar a una separación segura; γ es la tasa de colisión desde el estado de riesgo. Las tres tasas son no negativas y cada fila de Q suma cero. Se supone que no hay salto directo de S a C; si la definición del estado seguro lo permite, debe añadirse esa transición.

Con tasas constantes, la matriz de probabilidades de transición después de un intervalo t se obtiene mediante la exponencial matricial. Para un inicio en S, la entrada correspondiente a C expresa la probabilidad de que la primera intersección ya haya ocurrido, dado que C es absorbente.

$$P(t) = \exp(t \cdot Q)$$

$$\Pr(\text{intersección hasta } t \mid \text{inicio en S}) = P_{SC}(t)$$

La misma rapidez del agente y los enemigos no determina α , β y γ ; también intervienen sus direcciones, la distancia y la estructura del laberinto. Una aproximación continua puede asignar relojes exponenciales independientes con la misma tasa de salto $\lambda = v / \ell$, donde v es la rapidez media mientras se permite avanzar y ℓ la longitud de una celda. Esto iguala la frecuencia media de movimiento, no una velocidad física constante; un agente bloqueado puede permanecer inmóvil.

Las tasas pueden estimarse dividiendo el número de transiciones observadas entre el tiempo total en el estado de origen. La clasificación S/R debe definirse antes de medir. El modelo agregado de tres estados es una aproximación porque omite la geometría. Para una descripción más precisa, cada estado debe incluir las posiciones conjuntas del agente y todos los enemigos; las tasas dependen de sus movimientos permitidos y de la política BFS. El modelo continuo es una propuesta analítica y no una función implementada ni una probabilidad medida en esta práctica.

Conclusión

La ejecución documentada alcanzó la meta en el paso 62 con semilla 42. BFS permite recalcular rutas ante cambios en las posiciones de los enemigos, pero su costo se acumula con cada búsqueda y su política puede mantener inmóvil al agente cuando no existe una conexión segura. Para escenarios mayores conviene evitar copiar rutas completas y evaluar la búsqueda incremental. El modelo de Markov permite expresar el riesgo de intersección, siempre que sus tasas se definan y estimen con datos adecuados.

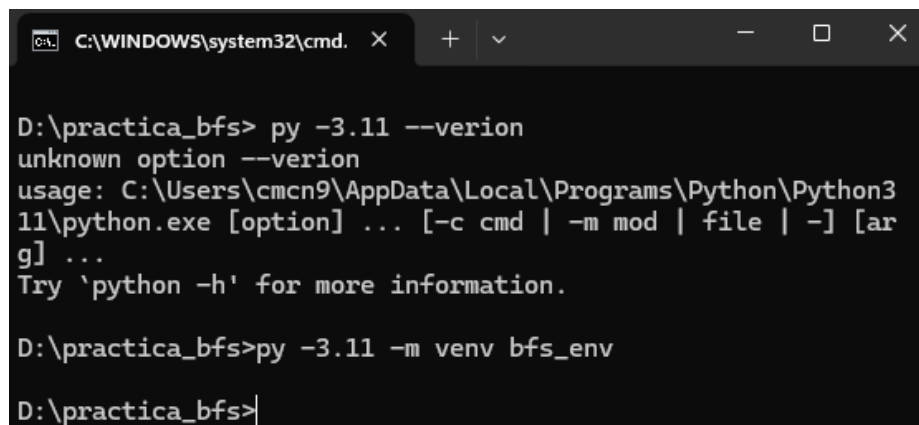
Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms (3.^a ed.). MIT Press.
- Koenig, S., & Likhachev, M. (2002). D* Lite. Proceedings of the AAAI Conference on Artificial Intelligence, 476–483. <https://idm-lab.org/bib/abstracts/Koen02e.html>
- Russell, S., & Norvig, P. (2020). Artificial intelligence: A modern approach (4.^a ed.). Pearson.
- Whitt, W. (2006). Continuous-time Markov chains [Material docente]. Columbia University. <https://www.columbia.edu/~ww2040/3106F12/CTMCchapter121906.pdf>

Archivos de entrega

La actividad solicita tres archivos separados: `automata_adversarial.ipynb` con el código funcional y la integración ejecutada localmente; un video .mp4 que muestre los cambios de ruta ante los enemigos; y este reporte en PDF. Las capturas complementan la evidencia, pero no sustituyen el notebook ni el video.

Anexo de preparación del entorno



```

C:\WINDOWS\system32\cmd. x + - □ x

D:\practica_bfs> py -3.11 --verion
unknown option --verion
usage: C:\Users\cmcn9\AppData\Local\Programs\Python\Python3
11\python.exe [option] ... [-c cmd | -m mod | file | -] [ar
g] ...
Try 'python -h' for more information.

D:\practica_bfs>py -3.11 -m venv bfs_env

D:\practica_bfs>
    
```

Figura 2. Creación del entorno bfs_env en la terminal de Windows.

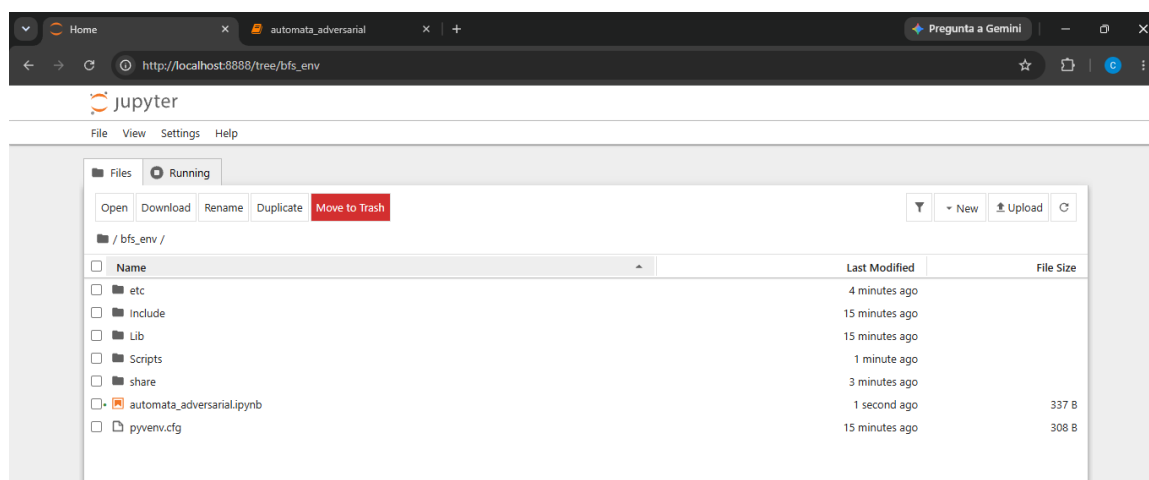


Figura 3. Vista de Jupyter y archivo automata_adversarial.ipynb. La captura muestra el notebook dentro de bfs_env; para organizar el proyecto conviene conservarlo fuera de la carpeta del entorno.

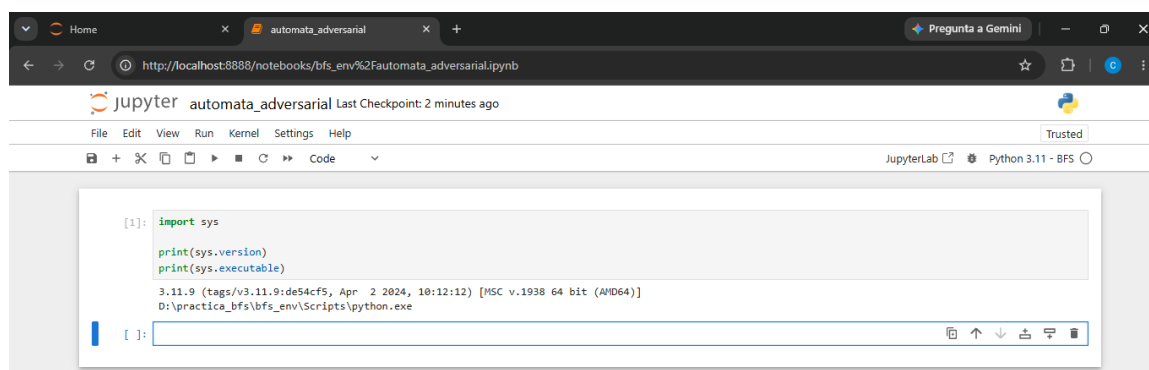


Figura 4. Verificación de Python 3.11.9 y del ejecutable del entorno virtual desde el notebook.

Anexo de asistencia de IA

Se consultó a la IA con el prompt estructurado indicado en la actividad. Las siguientes capturas conservan la consulta y fragmentos de las respuestas. El análisis del cuerpo del reporte distingue las propuestas teóricas de los resultados visibles en la ejecución.

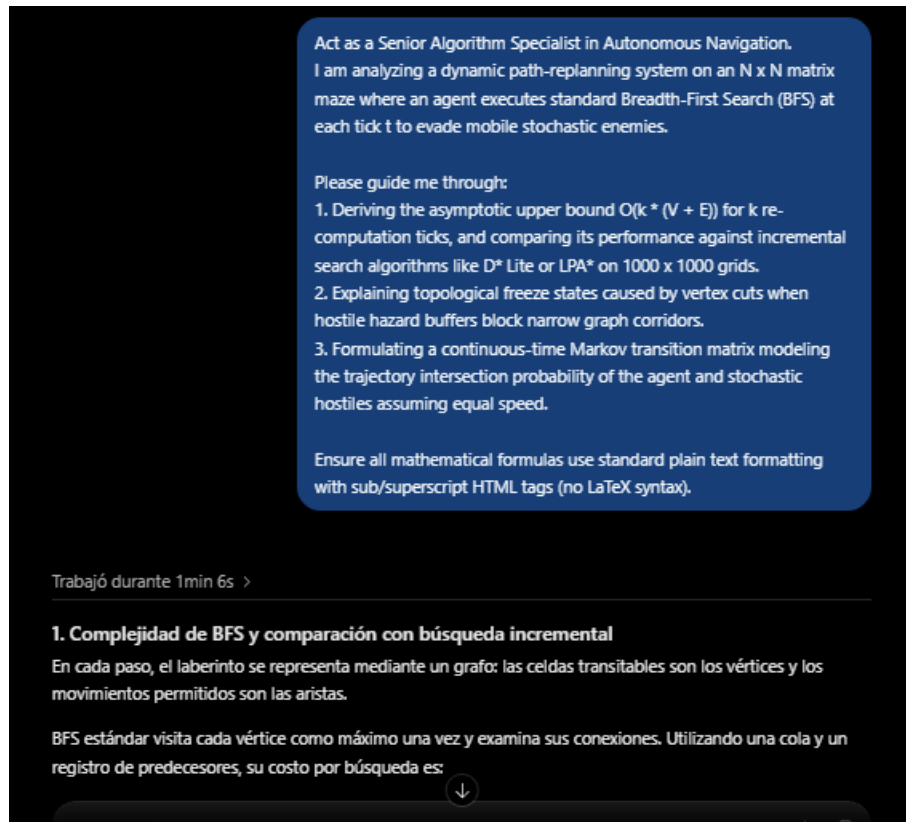


Figura 5. Prompt de asistencia sobre complejidad, cortes de vértices y modelo de Markov.

Evidencias de las preguntas críticas

- Complexity Bound:** Prove the asymptotic upper bound $O(k \cdot (|V| + |E|))$ for BFS re-computation across k frames. Discuss the feasibility of this model on 1000×1000 grids compared to incremental graph search algorithms (e.g., D* Lite).

BFS estándar agrega cada vértice a la cola como máximo una vez y examina cada arista un número constante de veces. Por ello, utilizando listas de adyacencia y predecesores, una búsqueda cuesta:

```

</> Texto sin formato
Tsub>BFS</sub> = O(|V| + |E|)
        
```

Si se recalcula desde cero durante k fotogramas, el trabajo total es la suma de las búsquedas. Tomando $|V|$ y $|E|$ como los tamaños máximos durante la simulación:

```

</> Texto sin formato
Tsub>total</sub>
≤ Tsub>BFS</sub> · k
= O(k · (|V| + |E|))
Tsub>total</sub> = O(k · (|V| + |E|))
        
```

En una cuadrícula de 1000×1000 , hay hasta un millón de vértices. Como cada celda tiene como máximo cuatro vecinos, $|E|$ es $O(|V|)$. Así, una búsqueda puede procesar una cantidad de información proporcional a un millón de celdas; repetirla en cada fotograma puede dificultar la ejecución en tiempo real.

Figura 6. Consulta y respuesta sobre la cota de complejidad de BFS.

- Topological Enclosure:** Explain the structural anomaly when hostile entities block all graph bottlenecks (vertex cuts). Why does BFS induce static inactivity (freezing in place) rather than maximizing spatial distance from enemies?

Cuando los enemigos y sus zonas de seguridad bloquean los pasos estrechos del laberinto, pueden formar un corte de vértices: un conjunto de celdas cuya eliminación desconecta la posición del agente de la meta. Aunque exista un camino en el laberinto original, este deja de existir en el grafo disponible para la navegación segura.

La inmovilidad ocurre por la combinación de esa desconexión y la lógica del programa. Si BFS no encuentra una ruta, la función devuelve `[start]`. Como el agente únicamente avanza cuando `len(path) > 1`, permanece en su posición y vuelve a intentar la búsqueda en el siguiente instante. Si el bloqueo continúa, también continúa la inmovilidad.

BFS busca el camino con menos pasos hacia una meta; no intenta maximizar la distancia respecto de los enemigos. Por eso, cuando la meta resulta inaccesible, el programa no dispone de un objetivo alternativo de evasión.

Para reducir este comportamiento, se puede añadir una estrategia de respaldo: cuando no exista una ruta a la meta, elegir entre las celdas vecinas permitidas aquella que maximice la distancia al enemigo más cercano. También conviene considerar la conectividad del espacio de escape, porque alejarse momentáneamente podría llevar al agente a un callejón sin salida.

Figura 7. Consulta y respuesta sobre bloqueo topológico e inmovilidad.

Evidencia de la propuesta probabilística

- Markov Transition Matrix:** Propose a transition matrix modeling the trajectory intersection probability in continuous time, assuming equivalent relative velocities for the agent and hostiles.

Trabajó durante 22s >

Se propone una cadena de Markov en tiempo continuo con tres estados: S, separación segura; R, proximidad de riesgo; y C, intersección o colisión. El estado C es absorbente, porque representa que el encuentro ya ocurrió.

La matriz generadora Q contiene tasas de transición por unidad de tiempo, no probabilidades:

Q	S	R	C
S	$-\alpha$	α	0
R	β	$-(\beta + \gamma)$	γ
C	0	0	0

Aquí, α representa la tasa de acercamiento; β , la tasa de regreso a una separación segura; y γ , la tasa de colisión desde la zona de riesgo. Las tasas son no negativas y cada fila suma cero.

Figura 8. Consulta y respuesta con una matriz generadora de Markov. La matriz corresponde a una propuesta analítica, no a parámetros estimados en la ejecución.